

OPERATIONAL RESEARCH & OPTIMISATION

Dr. Fang He

Email: hef@westminster.ac.uk

Office: N7.118

University of Westminster

Outline

- Integer Programming
 - General Integer Programming model techniques
 - Packing Optimisation Problems
 - Capacity and Indicator Variables
 - Knapsack Type Problems
 - Generalised assignment problems
 - Bin Packing Type Problems
 - Routing and location problems
 - Travelling Salesman Problem
 - Facility location problem

Integer Programming

Discrete optimization refers in general to those optimization problems (linear or non-linear) involving discrete decision variables, i.e. of the following types:

- Integer Programming
- Binary Integer Programming
- Mixed-Integer Programming

In combinatorial optimization problems (COP) the goal is to find the optimal setting of a set of discrete entities such that given requirements and perhaps constraints are satisfied. The optimal setting can be an arrangement, ordering, grouping, selection, distribution, etc.

Sometimes it might not be possible to model COPs as linear models, so they have to be tackled as non-linear problems.

Need for Integer Decision Variables

- Problems with [discrete input/outputs](#)
- Problems with [logical conditions](#)
- Many cases of [combinatorial optimization](#) problems
- Few hundreds general integer variables makes a problem [computationally difficult](#) to solve
- Consider building a [smaller version](#) of the model
- Using [reformulations](#) can help to reduced difficulty
- Practical solutions might be obtained with [heuristic methods](#)
- Sensitivity analysis is only possible for LP models but some [tricks can be used](#) for models with discrete variables

The advantages of integer programs

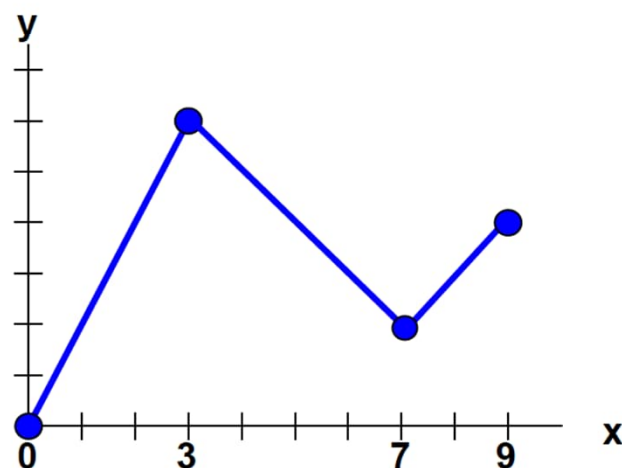
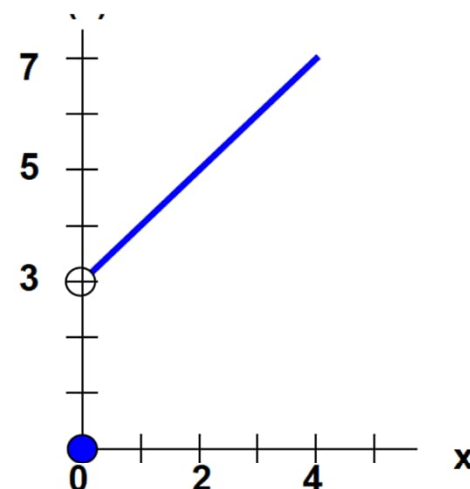
- **Rule of thumb:** integer programming can model any of the variables and constraints that you really want to put into an LP, but can't.
- **Binary variables**
 - $x_i = 1$ if we decide to do project i (else, it is 0)
 - $x_i = 1$ if node i is selected in the graph (else 0)
 - $x_{ij} = 1$ if we carry out task j after task i (else, 0)
 - $x_{it} = 1$ if we take subject i in semester t (else, 0)

Examples of constraints

- If project i is selected, then project j is not selected.
- If $x_1 > 0$, then $x_1 \geq 10$.
- $x_3 \geq 5$ or $x_4 \geq 8$.
- x_1, x_2, x_3, x_4, x_5 , are all different integers in $\{1, 2, 3, 4, 5\}$
- x is divisible by 7
- x is either 1 or 2 or 4 or 8 or 32

Nonlinear functions can be modeled using integer programming

$$f(x) = \begin{cases} 3 + x & \text{if } x > 0 \\ 0 & \text{if } x = 0 \end{cases}$$



$$y = 2x \quad \text{if } 0 \leq x \leq 3$$

$$y = 9 - x \quad \text{if } 3 \leq x \leq 7$$

$$y = -5 + x \quad \text{if } 7 \leq x \leq 9$$

Integer Programs

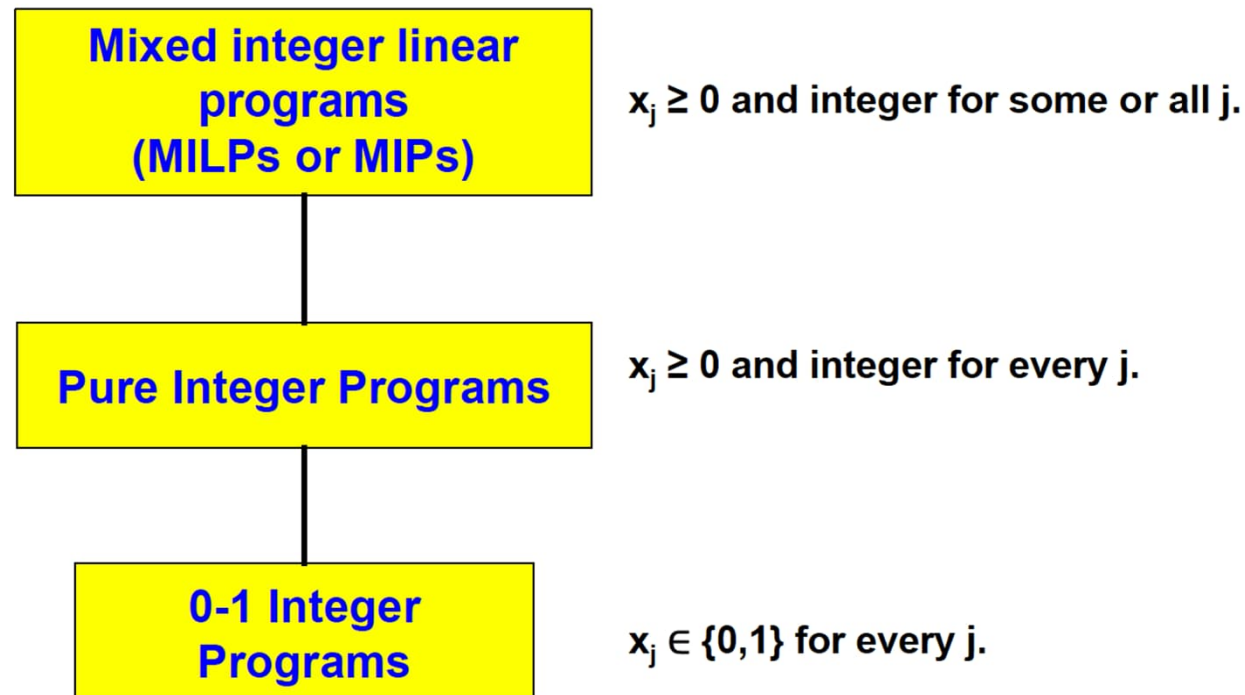
Integer programs: a linear program plus the additional constraints that some or all of the variables must be integer valued.

We also permit “ $x_j \in \{0,1\}$,” or equivalently, “ x_j is **binary**”

This is a shortcut for writing the constraints:

$$0 \leq x_j \leq 1 \text{ and } x_j \text{ integer.}$$

Types of Integer Programs



Note, pure integer programming instances that are unbounded can have an infinite number of solutions. But they have a finite number of solutions if the variables are bounded.

Integer Programming

Difficulty of LP, IP, BIP and MIP problems

- LP problems are in general easier to solve than IP and BIP problems.
- BIP problems are considered easier to solve than IP problems.
- The size of IP and BIP problems grows exponentially with the number of decision variables.
- The [LP-relaxation](#) is often used to solve IP problems.
- The non-integer (i.e. fractional) decision variables in MIP problems do not have much effect of the difficulty of the problem.

Integer Programming

Many Applications of BIP Models

- Invest or not on that company?
- Assign or not a person to that timeslot?
- Build or not a road between two cities?
- Include or not that site in the tour?
- Deliver or not this product with that truck?
- Schedule or not this match in premium TV time?
- Assign or not this airport gate to that aircraft?
- Produce or not that new article in this factory?
- Include or not this item in my luggage?
- Should activity A follow or precede activity B?

A 2-Variable Integer program

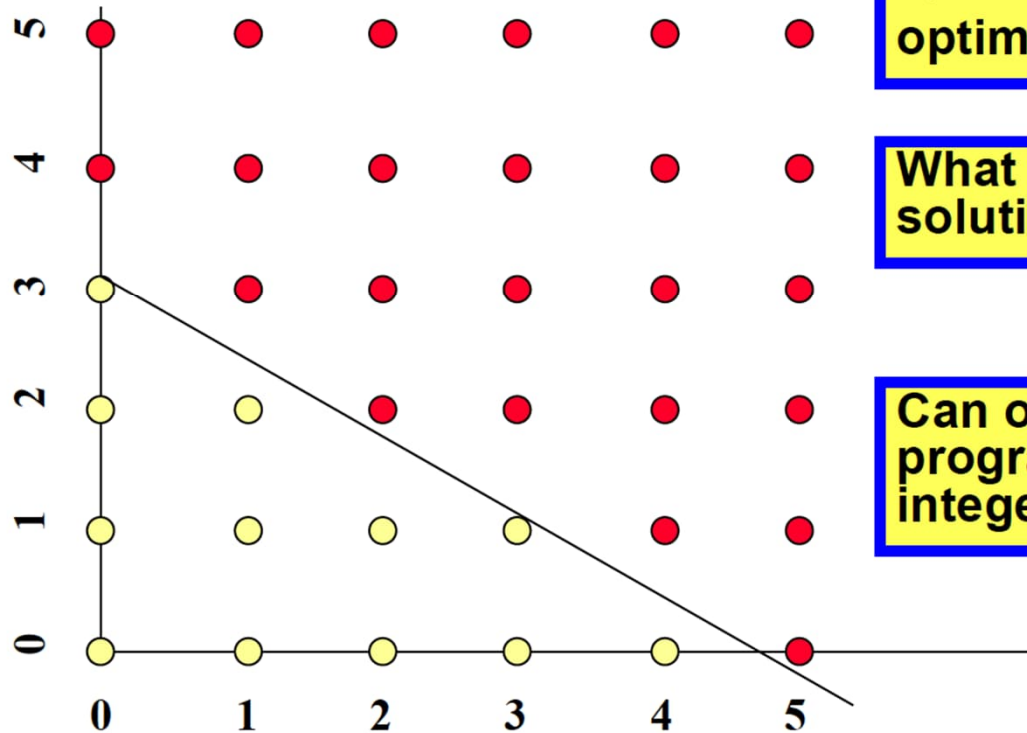
maximize **$3x + 4y$**

subject to **$5x + 8y \leq 24$**

$x, y \geq 0$ and integer

- **What is the optimal solution?**

The Feasible Region

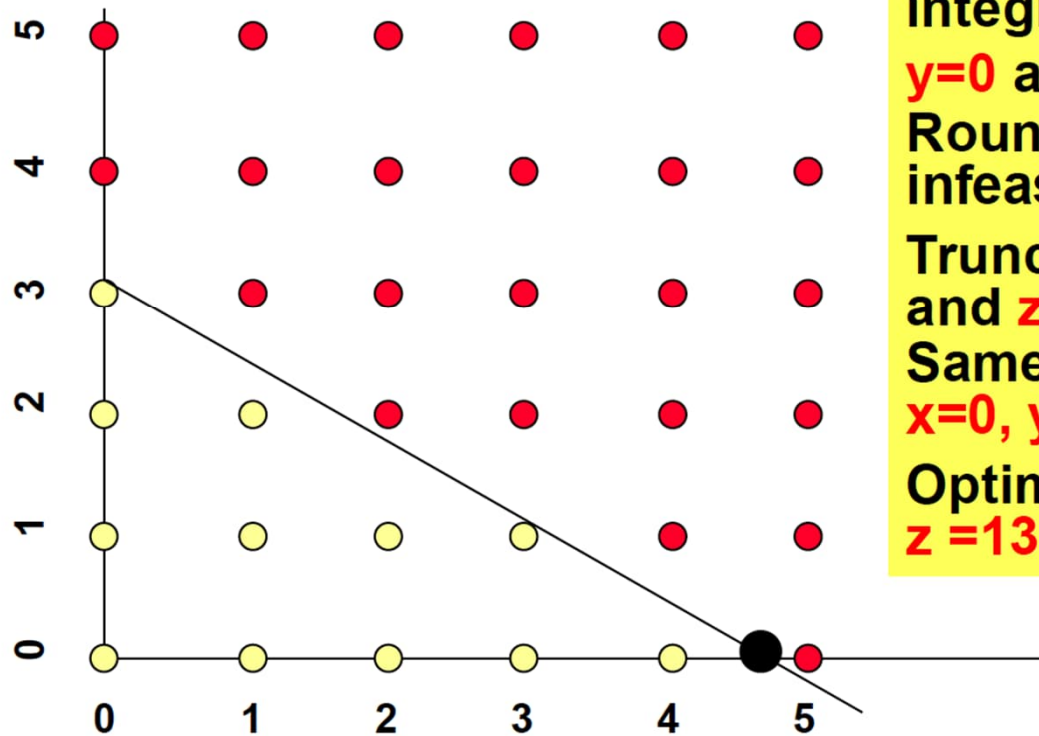


Question: What is the optimal integer solution?

What is the optimal linear solution?

Can one use linear programming to solve the integer program?

A rounding technique that sometimes is useful, and sometimes not.



Solve LP (ignore integrality) get $x=24/5$, $y=0$ and $z=14\frac{2}{5}$.

Round, get $x=5$, $y=0$, infeasible!

Truncate, get $x=4$, $y=0$, and $z=12$

Same solution value at $x=0$, $y=3$.

Optimal is $x=3$, $y=1$, and $z=13$

How integer programs are solved (and why they are hard to solve)

- Rely on solving LPs fast
- Branch and bound and cutting planes
- Out of scope of this module

Why integer programs?

- **Advantages of restricting variables to take on integer values**
 - **More realistic**
 - **More flexibility**
- **Disadvantages**
 - **More difficult to model**
 - **Can be much more difficult to solve**

Some Comments on IP models

- There are often multiple ways of modeling the same integer program.
- Solvers for integer programs are extremely sensitive to the formulation. (not true for LPs).
- Not as easy to model
- Not as easy to solve.

Some IP model techniques

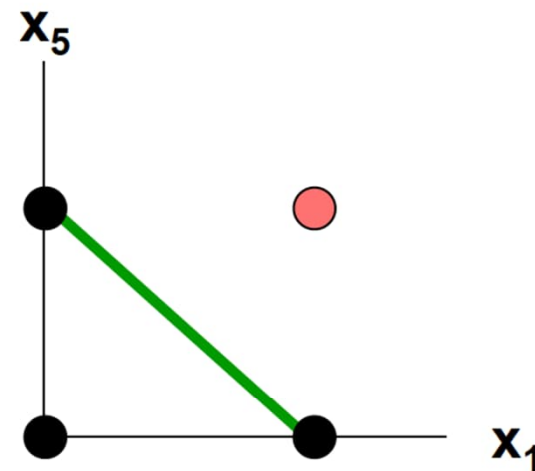
- Modeling logical constraints that include only two binary variables.
 - Step 1. Graph the feasible region as restricted to the two variables.
 - Step 2. Add linear equalities and or inequalities so that the feasible region of the IP is the same as that given in Step 1.

Logical Constraints 1

Constraint 1. If you select the iPad, you cannot select iPhone

$x_1 = 1$ if iPad

$x_5 = 1$ if iPhone



MIP Constraint:

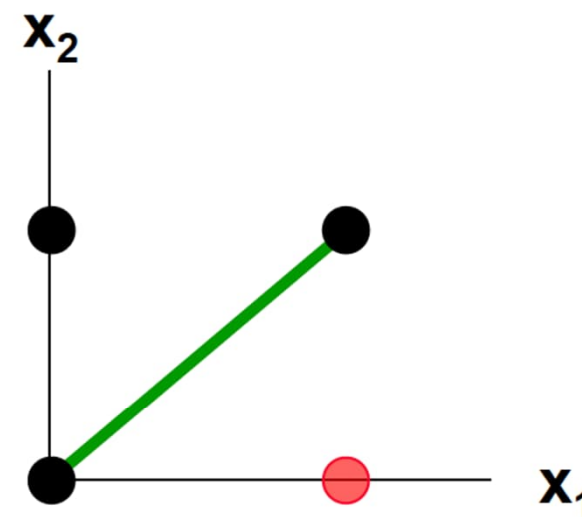
$$x_1 + x_5 \leq 1$$

Logical Constraints 2

Constraint 2. If Prize 1 is selected then Prize 2 must be selected.

$x_1 = 1$ if Prize 1

$x_2 = 1$ if Prize 2



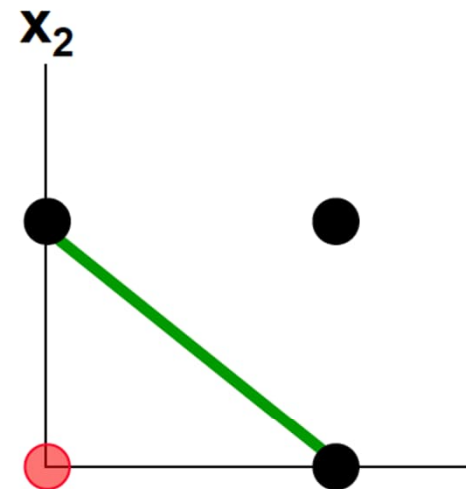
MIP Constraint: $x_1 \leq x_2$

Logical Constraints 3

Constraint 3. You must select Prize 1 or Prize 2 or both

$x_1 = 1$ if Prize 1

$x_2 = 1$ if Prize 2



MIP Constraint:

$$x_1 + x_2 \geq 1$$

Other logical constraints

- Modeling logical constraints that involve non-binary variables is much harder.
- BIG M Method for IP Formulations.

BIG M Method for IP Formulations

- Assume that all variables are integer valued.
- Assume a bound u^* on coefficients and variables;
 - e.g., $x_j \leq 10,000$ for all j .
 $|a_{ij}| \leq 10,000$ for all i, j .
- Choose M really large so that for every constraint i ,

$$|a_{i1}x_1 + a_{i2}x_2 + \dots + a_{in}x_n| \leq b_i + M$$

That is, we will be able to satisfy any “ $\leq$ ” constraint by adding M to the RHS.

And we can satisfy any “ $\geq$ ” constraint by subtracting M from the RHS.

The logical constraint “ $x \leq 2$ or $x \geq 6$ ”

We formulate the logical constraint,
“ $x \leq 2$ or $x \geq 6$ ” as follows.

Choose a binary variable w so that

if $w = 1$, then $x \leq 2$.

if $w = 0$, then $x \geq 6$.

$$x \leq 2 + M(1-w)$$

$$x \geq 6 - M w$$

$$w \in \{0,1\}$$

To validate the formulation one needs to show: The logical constraints are equivalent to the IP constraints.

Suppose that (x, w) is feasible, for the IP.

If $w = 1$, then $x \leq 2$.

If $w = 0$, then $x \geq 6$.

Using Excel Solver to Solve Integer Programs

- **Add the integrality constraints (or add that a variable is binary)**
- **Set the Solver Tolerance. (Integer optimality %)**
(The tolerance is the percentage deviation from optimality allowed by solver in solving Integer Programs.)
 - **The default used to be 5%.**
 - **A 5% default is way too high**
 - **It often finds the optimum for small problems**

A general example 1

Fixed-charge Problems

Select the best mix of competing activities given limited resources. The total cost of each activity is given by the cost due to the activity level plus a fixed-charge cost.

Internet Connection Problem

A customer needs at least 3000 minutes of INTERNET CONNECTION every month and can choose the services of any combination from three companies A,B,C. The problem is to minimise the overall cost by deciding how many minutes (must be integer number) to contract from each company. Company A charges a £10 connection fee plus 5 cents per minute. Company B charges a £20 connection fee plus 3 cents per minute. Company C charges a £25 connection fee plus 2.5 cents per minute.

A general example-1

x_1, x_2, x_3 are minutes from company A, B, C respectively

y_1, y_2, y_3 indicate if service from company A, B, C is used

Minimise: $Z = 0.05x_1 + 0.03x_2 + 0.025x_3 + 10y_1 + 20y_2 + 25y_3$

Subject to: $x_1 + x_2 + x_3 \geq 3000$ (1)

$x_1 \leq My_1$ (2)

$x_2 \leq My_2$ (3)

$x_3 \leq My_3$ (4)

$x_1, x_2, x_3 \geq 0$ and integer

$y_1, y_2, y_3 \in \{0,1\}$

$M = 3000$

Minimum Cost: 100

Knapsack Problems

Knapsack Problem

Given a knapsack of given capacity and a set of N items each of given size s_i and profit p_i the problem is to select which items to pack in the knapsack so that the total profit is maximised without exceeding the capacity B of the knapsack.

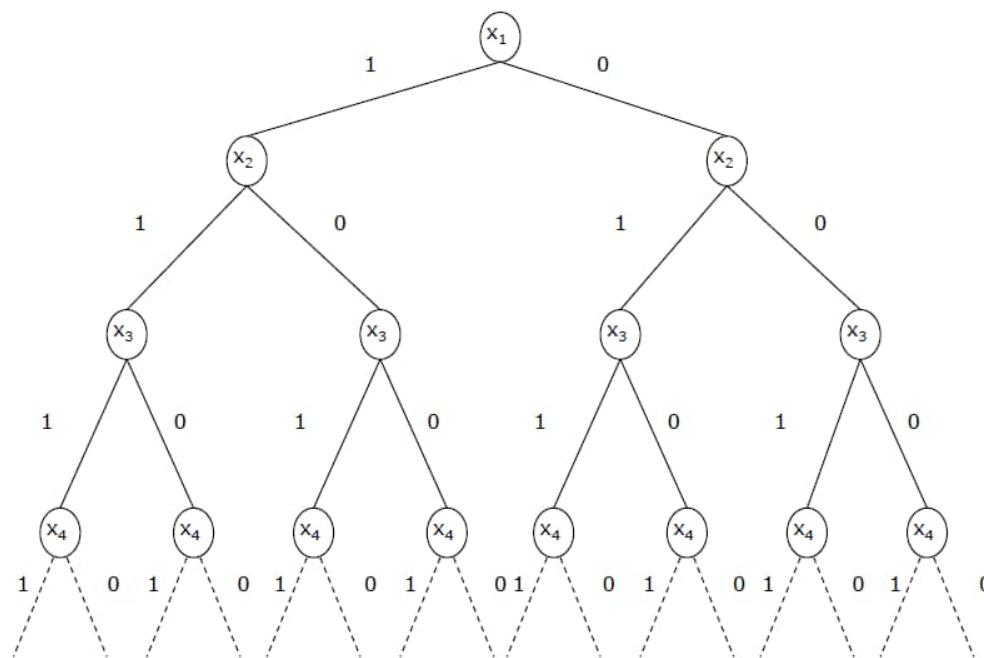
$$\begin{array}{ll}\text{Maximise :} & Z = \sum_{i=1}^N p_i x_i \\ \text{Subject to:} & \sum_{i=1}^N s_i x_i \leq B \quad (1) \\ & x_i = 1 \text{ if item } i \text{ is packed, } 0 \text{ otherwise}\end{array}$$

Sketch of the Knapsack Problem (N=4)

$$\begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ \vdots \\ p_{N-1} \\ p_N \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-1} \\ x_N \end{bmatrix} = Z$$

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_{N-1} \\ s_N \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-1} \\ x_N \end{bmatrix} \leq B$$

There are 2^N ways of selecting a subset from the set of N items.
This includes the empty set.



Multiple Knapsack Problem

Given a set of M knapsacks each of given capacity and a set of N items each of given size s_i and profit p_{ij} (the profit depends on which knapsack the item is packed into), the problem is to select which items to pack into each of the knapsacks so that the total profit is maximised without exceeding the capacity B_j of any of the knapsacks.

$$\text{Maximise : } Z = \sum_{j=1}^M \sum_{i=1}^N p_{ij} x_{ij}$$

$$\text{Subject to: } \sum_{i=1}^N s_i x_{ij} \leq B_j \quad \text{for } j = 1, 2, \dots, M \quad (1)$$

$$\sum_{j=1}^M x_{ij} \leq 1 \quad \text{for } i = 1, 2, \dots, N \quad (2)$$

$$x_{ij} = 1 \text{ if item } i \text{ is packed into knapsack } j, 0 \text{ otherwise}$$

Sketch of the Multiple Knapsack Problem

$$\begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1M} \\ p_{21} & p_{22} & \cdots & p_{2M} \\ p_{31} & p_{32} & \cdots & p_{3M} \\ \vdots & \vdots & \vdots & \vdots \\ p_{N1} & p_{N2} & \cdots & p_{NM} \end{bmatrix} \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1M} \\ x_{21} & x_{22} & \cdots & x_{2M} \\ x_{31} & x_{32} & \cdots & x_{3M} \\ \vdots & \vdots & \vdots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{NM} \end{bmatrix} = Z$$

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_N \end{bmatrix} \begin{bmatrix} x_{11} \\ x_{21} \\ x_{31} \\ \vdots \\ x_{N1} \end{bmatrix} \leq B_1 \quad \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_N \end{bmatrix} \begin{bmatrix} x_{12} \\ x_{22} \\ x_{32} \\ \vdots \\ x_{N2} \end{bmatrix} \leq B_2 \quad \cdots \quad \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_N \end{bmatrix} \begin{bmatrix} x_{1M} \\ x_{2M} \\ x_{3M} \\ \vdots \\ x_{NM} \end{bmatrix} \leq B_M$$

$$\begin{aligned} [x_{11} + x_{12} + x_{13} + \cdots + x_{1M}] &\leq 1 \\ [x_{21} + x_{22} + x_{23} + \cdots + x_{2M}] &\leq 1 \\ [x_{31} + x_{32} + x_{33} + \cdots + x_{3M}] &\leq 1 \\ \vdots &\vdots \\ [x_{N1} + x_{N2} + x_{N3} + \cdots + x_{NM}] &\leq 1 \end{aligned}$$

Generalised Assignment Problem

Given a set of N tasks and a set of M workers with a certain cost c_{ij} for assigning each task i to each worker j . Each worker has a limited amount of time available T_j , each task i takes a given worker j a certain amount of time t_{ij} . Each task must be assigned to exactly one worker but a worker can undertake more than one task according to the time available. The problem is to assign all the tasks to the workers so that the total cost is minimised without exceeding the time available of any worker.

Minimise : $Z = \sum_{j=1}^M \sum_{i=1}^N c_{ij} x_{ij}$ Subject to: $\sum_{j=1}^M x_{ij} = 1 \quad \text{for } i = 1, 2, \dots, N \quad (1)$ $\sum_{i=1}^N t_{ij} x_{ij} \leq T_j \quad \text{for } j = 1, 2, \dots, M \quad (2)$ $x_{ij} = 1$ if task i is assigned to worker j, 0 otherwise
--

Sketch of the Generalised Assignment Problem

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1M} \\ c_{21} & c_{22} & \cdots & c_{2M} \\ c_{31} & c_{32} & \cdots & c_{3M} \\ \vdots & \vdots & \vdots & \vdots \\ c_{N1} & c_{N2} & \cdots & c_{NM} \end{bmatrix} \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1M} \\ x_{21} & x_{22} & \cdots & x_{2M} \\ x_{31} & x_{32} & \cdots & x_{3M} \\ \vdots & \vdots & \vdots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{NM} \end{bmatrix} = Z$$

$$\begin{aligned} [x_{11} + x_{12} + x_{13} + \cdots + x_{1M}] &= 1 \\ [x_{21} + x_{22} + x_{23} + \cdots + x_{2M}] &= 1 \\ [x_{31} + x_{32} + x_{33} + \cdots + x_{3M}] &= 1 \\ \vdots & \quad \quad \quad \vdots \\ [x_{N1} + x_{N2} + x_{N3} + \cdots + x_{NM}] &= 1 \end{aligned}$$

$$\begin{bmatrix} t_{11} \\ t_{21} \\ t_{31} \\ \vdots \\ t_{N1} \end{bmatrix} \begin{bmatrix} x_{11} \\ x_{21} \\ x_{31} \\ \vdots \\ x_{N1} \end{bmatrix} \leq T_1 \quad \begin{bmatrix} t_{12} \\ t_{22} \\ t_{32} \\ \vdots \\ t_{N2} \end{bmatrix} \begin{bmatrix} x_{12} \\ x_{22} \\ x_{32} \\ \vdots \\ x_{N2} \end{bmatrix} \leq T_2 \quad \cdots \quad \begin{bmatrix} t_{1M} \\ t_{2M} \\ t_{3M} \\ \vdots \\ t_{NM} \end{bmatrix} \begin{bmatrix} x_{1M} \\ x_{2M} \\ x_{3M} \\ \vdots \\ x_{NM} \end{bmatrix} \leq T_M$$

Packing optimisation problems

Capacity and Indicator Variables

- Some problems involve selecting items for packing them into containers of limited capacity.
- The most common constraint is the limited capacity of the containers available.
- In some packing problems, besides binary variables to decide whether an item is selected or not for packing, indicator binary variables are also needed to indicate the use or not of some containers.

Indicator Variables

This technique helps to 'link' variables for dependency:

$$x - My \leq 0$$

Indicator variables, means: If $x > 0$ then $y = 1$

$$y - x \leq 0$$

Indicator variables, means: If $x = 0$ then $y = 0$

Bin-Packing Problems

Bin Packing Problem

Given a set of M bins all of equal capacity B and a set of N items each of given size s_i the problem is to pack all the items into bins so that the total number of bins used is minimised without exceeding the capacity of any bin.

$$\text{Minimise : } Z = \sum_{j=1}^M y_j$$

$$\text{Subject to: } \sum_{i=1}^N s_i x_{ij} \leq y_j B \quad \text{for } j = 1, 2, \dots, M \quad (1)$$

$$\sum_{j=1}^M x_{ij} = 1 \quad \text{for } i = 1, 2, \dots, N \quad (2)$$

$x_{ij} = 1$ if item i is packed into bin j , 0 otherwise

$y_j = 1$ if bin j is used, 0 otherwise

Sketch of the Bin-Packing Problem

$$[y_1 \ y_2 \ y_3 \ \cdots \ y_M] = Z$$

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_N \end{bmatrix} \begin{bmatrix} x_{11} \\ x_{21} \\ x_{31} \\ \vdots \\ x_{N1} \end{bmatrix} \leq y_1 B \quad \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_N \end{bmatrix} \begin{bmatrix} x_{12} \\ x_{22} \\ x_{32} \\ \vdots \\ x_{N2} \end{bmatrix} \leq y_2 B \quad \cdots \quad \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_N \end{bmatrix} \begin{bmatrix} x_{1M} \\ x_{2M} \\ x_{3M} \\ \vdots \\ x_{NM} \end{bmatrix} \leq y_M B$$

$$[x_{11} + x_{12} + x_{13} + \cdots + x_{1M}] = 1$$

$$[x_{21} + x_{22} + x_{23} + \cdots + x_{2M}] = 1$$

$$[x_{31} + x_{32} + x_{33} + \cdots + x_{3M}] = 1$$

$$\vdots$$

$$[x_{N1} + x_{N2} + x_{N3} + \cdots + x_{NM}] = 1$$

Some example applications of IP-1

Some Example Applications

Project Budgeting Problem

For PROJECT BUDGETING, a manager must select the combination of 3-year projects between four of them so as to maximise return. The expenditures and returns for each project are given below. Additional conditions are that projects 1 and 3 cannot be chosen at the same time and project 4 can be chosen only if project 1 is chosen.

Project	Expenditure (millions £) per Year			Return (millions £)
	Y1	Y2	Y3	
1	5	1	8	20
2	4	7	10	40
3	6	9	3	20
4	7	4	2	15
Budget (millions £)	18	22	20	

Maximum return: 75

Some example applications of IP-2

Software Components Problem

The designers of a complex software system want to decide which SOFTWARE COMPONENTS to include in order to maximise the total value without exceeding the development budget of £5000. The value v_i and cost c_i associated to each component are given below.

Components 2 and 6 must be both implemented or none of them at all.

	1	2	3	4	5	6	7	8	9	10
Value	1200	600	2300	1100	980	3100	550	1800	400	850
Cost	800	300	1500	700	1000	1200	230	1100	150	560

**Maximum total
value: 9300**

Some example applications of IP-3

Capital Investments Problem

An investor is considering seven large CAPITAL INVESTMENTS. Each investment can be made only once. The profit p_i and required capital c_i for each investment are given. The total amount of capital available for these investments is denoted by B . Investments 1 and 2 are mutually exclusive, so are 3 and 4. Neither 3 and 4 can be taken unless one of the two first investments is chosen. The objective is to select the combination of investments that maximises the total profit.

Memory Card

Decision variables

$x_i \in \{0,1\}$	Yes/No decisions
$x_i \in \{0,1,2,\dots\}$	Not hire staff, hire staff type 1, hire staff type 2, etc.
$x - My \leq 0$	Indicator variables, means: If $x > 0$ then $y = 1$
$y - x \leq 0$	Indicator variables, means: If $x = 0$ then $y = 0$

Logical Conditions

Having x_i as binary variables then:

$x_1 + x_2 \leq 1$	Mutually exclusive alternatives	$x_1 + x_2 + x_3 + x_4 + x_5 \leq 3$
$x_1 + x_2 \geq 1$	At least one of the alternatives	$x_1 + x_2 + \dots + x_{n-1} + x_n \geq n/2$
$x_1 + x_2 = 1$	Exactly one of the alternatives	$x_1 + x_2 + x_3 + x_4 = 2$
$x_1 - x_2 \leq 0$	If-Then decision (contingent)	$x_1 \leq x_2 + x_3 \quad x_1 < x_2 + x_3$
$x_1 - x_2 = 0$	If -Only-If decision (coincide)	$x_1 + x_2 - x_3 - x_4 = 0$
$x_1 + x_2 = 2$	Both conditions must hold	$x_1 + x_2 + x_3 + x_4 = 4$

- For more **Integer programming formulation examples** see
- <http://people.brunel.ac.uk/~mastjjb/jeb/or/moreip.html>